

Project Submission – Database Administration

BANKING DATABASE

STUDENT DETAILS

Student Name	GOKUL NATH K.R
Register Number	2422J1144
Department	BACHELOR OF COMPUTER APPLICATIONS
ID	E9BCC3C7C71975FAD7D42DAD7BC629B8
College	Government Arts & Science College Gudalur
Date of Submission	23-09-2026

1. PROBLEM STATEMENT & OBJECTIVES

PROBLEM STATEMENT

A banking system needs to maintain customer information, account details, account types, deposits, withdrawals, and transaction history. Managing these records manually can cause duplicate data, calculation errors, difficulty tracking transactions, and problems while retrieving account information. The Banking Database is designed to store and manage these records in an organized and reliable manner using MySQL.

OBJECTIVES

- To store customer information in a structured database.*
- To maintain account details for each customer.*
- To manage different account types.*
- To record deposits and withdrawals as transactions.*
- To maintain complete transaction history.*
- To establish relationships using primary and foreign keys.*
- To maintain data accuracy and integrity using constraints.*
- To perform CRUD operations, filtering, sorting, grouping, aggregate functions, and joins using SQL.*

2. DATABASE DESIGN

2.1 DATABASE STRUCTURE

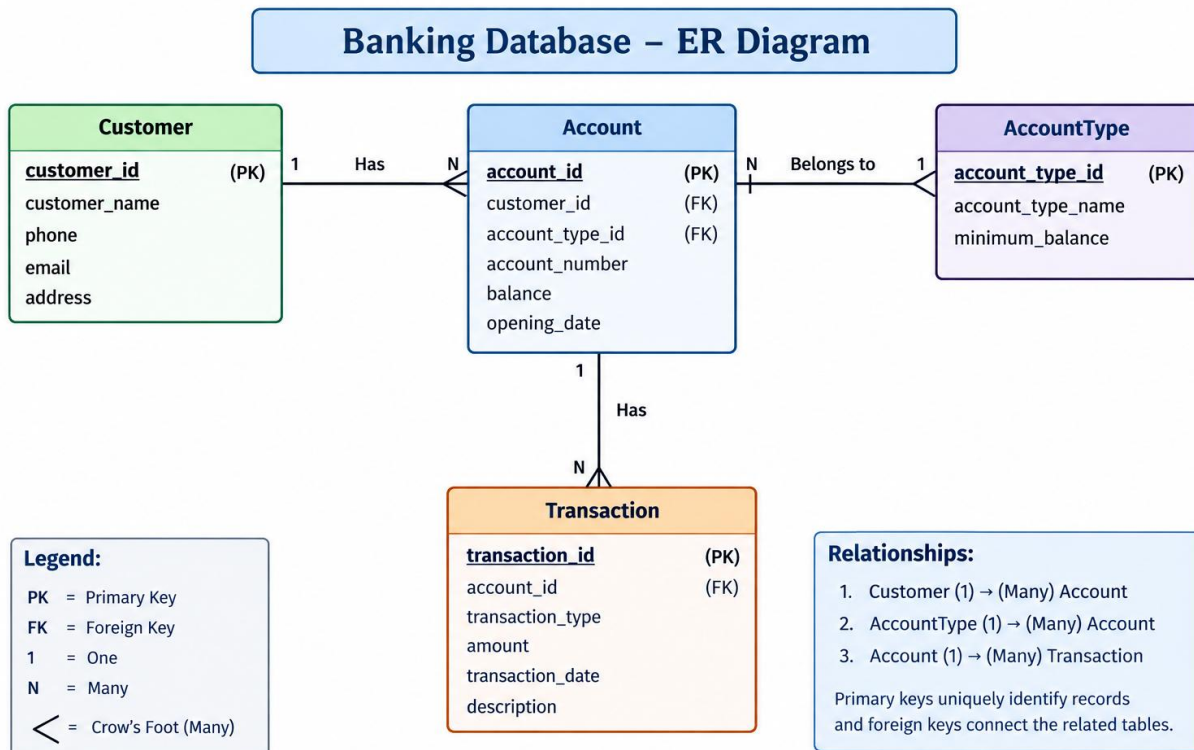
The database is named banking_db. Four related tables are used: Customer, AccountType, Account, and Transaction. AccountType stores controlled account categories, Account stores customer account details, and Transaction stores

deposits and withdrawals for each account.

Table Name	Purpose
Customer	Stores customer information
AccountType	Stores account type information
Account	Stores customer account details and balance
Transaction	Stores deposit, withdrawal, and transaction history

Tables Used

2.2 Entity Relationship Diagram



ER Diagram: Customer (1) → (Many) Account; AccountType (1) → (Many) Account; Account (1) → (Many) Transaction. Primary keys uniquely identify records and foreign keys connect the related tables.

Diagram:

ER Diagram: Customer → Account ← AccountType; Account → Transaction
(Insert your Banking Database ER diagram screenshot here)

2.3 Technologies Used

MySQL 8.0 – Database management system.
MySQL Workbench – Database design and SQL execution.
SQL – Database creation, data manipulation, querying, filtering, aggregation, and joins.

Technology / Tool	Purpose
MySQL	Database management system
MySQL Workbench	Database design and SQL execution
SQL	Database creation, manipulation, and querying

3. Practical Implementation

3.1 Database and Table Creation

Provide the SQL commands used to create the database and tables.

Database Creation

```
CREATE DATABASE IF NOT EXISTS banking_db;
USE banking_db;
```

Customer Table

```
CREATE TABLE Customer (
  customer_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_name VARCHAR(100) NOT NULL,
  phone VARCHAR(15) NOT NULL UNIQUE,
  email VARCHAR(100) UNIQUE,
  address VARCHAR(200)
);
```

Account Table

```
CREATE TABLE AccountType (
  account_type_id INT PRIMARY KEY,
  account_type_name VARCHAR(50) NOT NULL UNIQUE
);
```

```
CREATE TABLE Account (
  account_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT NOT NULL,
  account_type_id INT NOT NULL,
  account_number VARCHAR(20) NOT NULL UNIQUE,
  opening_date DATE NOT NULL,
  balance DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
  FOREIGN KEY (account_type_id) REFERENCES AccountType(account_type_id)
);
```

AccountType / Transaction Data

```
CREATE TABLE AccountType (
  account_type_id INT PRIMARY KEY,
  account_type_name VARCHAR(50) NOT NULL UNIQUE
);
```

```
CREATE TABLE Transaction (
  transaction_id INT PRIMARY KEY AUTO_INCREMENT,
  account_id INT NOT NULL,
  transaction_type VARCHAR(20) NOT NULL,
  amount DECIMAL(12,2) NOT NULL,
  transaction_date DATE NOT NULL,
  description VARCHAR(200),
  FOREIGN KEY (account_id) REFERENCES Account(account_id)
);
```

3.2 Constraints and Relationships

Primary keys uniquely identify each record. Foreign keys establish relationships between Customer, AccountType, Account, and Transaction. NOT NULL ensures important values are entered. UNIQUE prevents duplicate account numbers, phone numbers, and email addresses. AUTO_INCREMENT generates numeric IDs automatically. DEFAULT sets a standard transaction status when required.

3.3 Data Insertion

Sample records are inserted into Customer, AccountType, Account, and Transaction using INSERT statements. Deposits and withdrawals are stored in the Transaction table using transaction_type.

Customer Data

```
INSERT INTO Customer (customer_name, phone, email, address)
VALUES
('Arun Kumar S','9876543210','arun@gmail.com','Gudalur'),
('Priya Devi','9876543211','priya@gmail.com','Chennai'),
('Rahul Raj','9876543212','rahul@gmail.com','Coimbatore');
```

Account Table

-- Insert INSERT statements here

Transaction Data

```
INSERT INTO Transaction (account_id, transaction_type, amount,
transaction_date, description)
VALUES
(1,'Deposit',10000.00,'2026-09-05','Cash deposit'),
(1,'Withdrawal',3000.00,'2026-09-08','ATM withdrawal'),
(2,'Deposit',25000.00,'2026-09-10','Salary credit'),
(2,'Withdrawal',5000.00,'2026-09-12','Cash withdrawal'),
(3,'Deposit',8000.00,'2026-09-15','Cash deposit');
```

3.4 SQL Operations

```

INSERT INTO Customer (customer_name, phone, email, address) VALUES ('Arun
Kumar', '9876543210', 'arun@gmail.com', 'Gudalur');
SELECT * FROM Customer;
UPDATE Account SET balance = balance + 5000 WHERE account_id = 1;
DELETE FROM Transaction WHERE transaction_id = 10;
SELECT * FROM Transaction WHERE transaction_type='Deposit';
SELECT * FROM Account ORDER BY balance DESC;
SELECT * FROM Customer WHERE customer_name LIKE 'A%';
SELECT * FROM Transaction WHERE amount BETWEEN 1000 AND 10000;
SELECT COUNT(*) AS total_transactions, SUM(amount) AS total_amount,
AVG(amount) AS average amount, MAX(amount) AS maximum_amount,
MIN(amount) AS minimum amount FROM Transaction;
SELECT transaction_type, COUNT(*) AS total_transactions, SUM(amount) AS
total_amount FROM Transaction GROUP BY transaction_type;
SELECT c.customer_name, a.account_number, at.account_type_name,
t.transaction_type, t.amount, t.transaction_date FROM Customer c INNER JOIN
Account a ON c.customer_id=a.customer_id INNER JOIN AccountType at ON
a.account_type_id=at.account_type_id INNER JOIN Transaction t ON
a.account_id=t.account_id;
INSERT creates records, SELECT retrieves records, UPDATE modifies records,
and DELETE removes records. These form the basic CRUD operations.

```

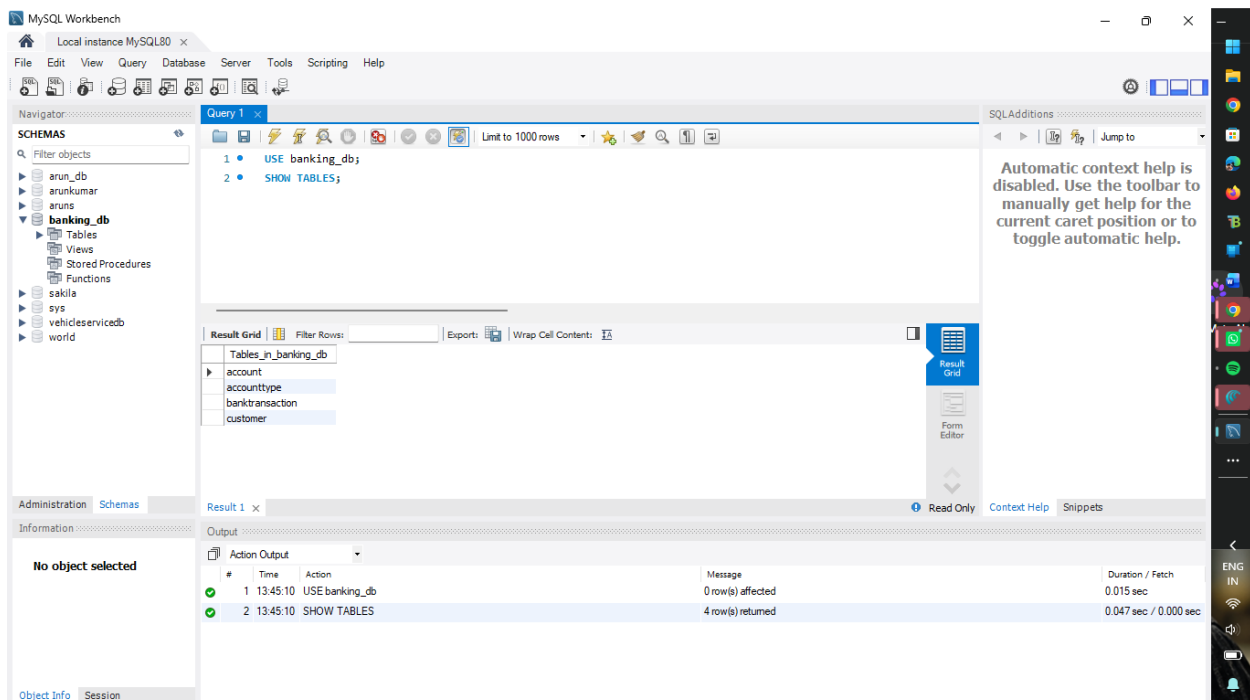
3.5 Project Execution

The project was implemented using MySQL Workbench. First, the `banking_db` database was created. Customer and AccountType tables were created before the dependent Account and Transaction tables. Sample records were inserted and the database was tested using `INSERT`, `SELECT`, `UPDATE`, `DELETE`, `WHERE`, `ORDER BY`, `LIKE`, `BETWEEN`, aggregate functions, `GROUP BY`, and `INNER JOIN`. The query results were verified in the MySQL Workbench output window.

4. Output Screenshots

Include screenshots showing the database creation, table structure, executed queries, and query results.

Screenshot 1 – Database and Tables



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the following structure:

- Filter objects
- arun_db
- arunkumar
- aruns
- banking_db**
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sakila
- sys
- vehideservice
- world

The main window shows the 'Query 1' editor with the following SQL:

```
1 • USE banking_db;
2 • SHOW TABLES;
```

The 'Result Grid' shows the output of the 'SHOW TABLES' query:

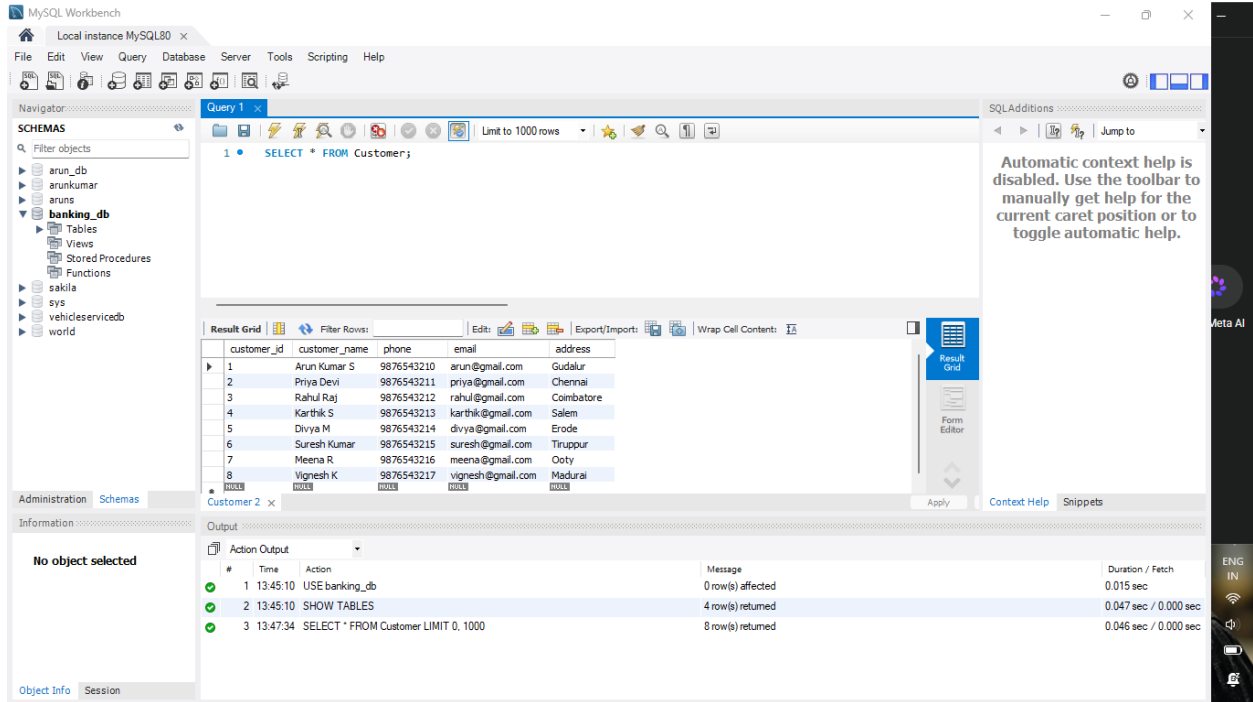
Tables_in_banking_db
account
accounttype
banktransaction
customer

The 'Output' window at the bottom shows the execution log:

#	Time	Action	Message	Duration / Fetch
1	13:45:10	USE banking_db	0 row(s) affected	0.015 sec
2	13:45:10	SHOW TABLES	4 row(s) returned	0.047 sec / 0.000 sec

Screenshot 2

Customer Table



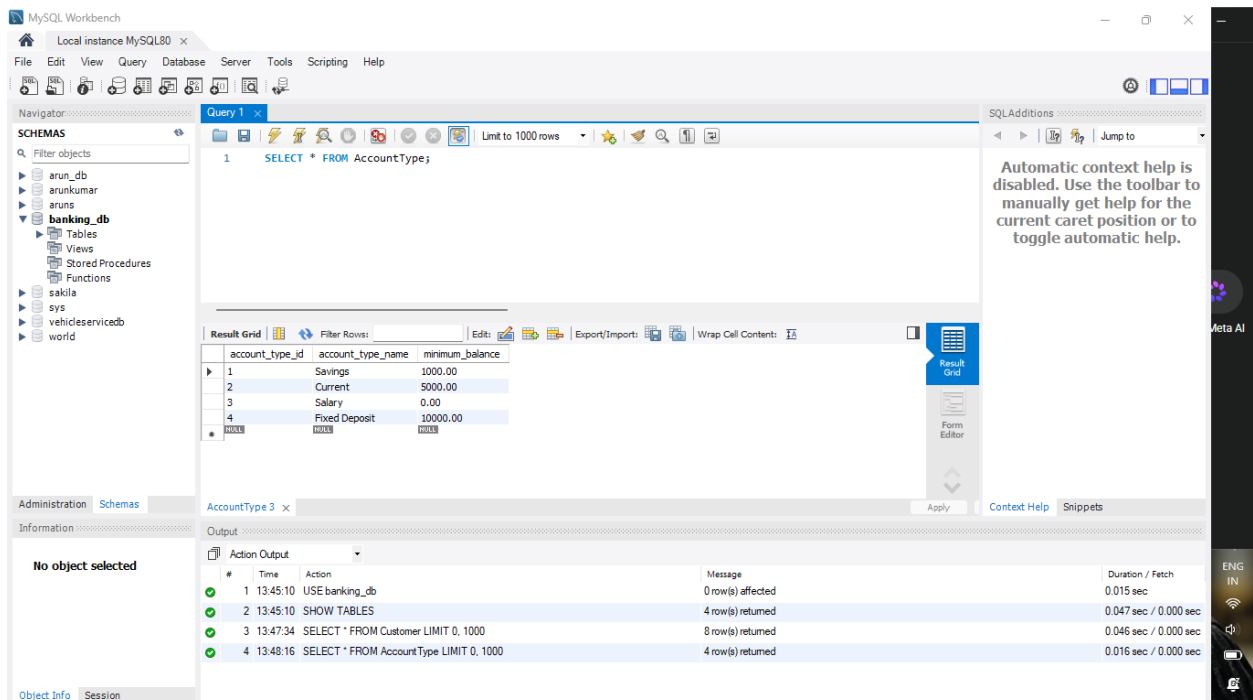
MySQL Workbench interface showing the Customer table data. The query executed is `SELECT * FROM Customer;`. The result grid displays 8 rows of customer information.

customer_id	customer_name	phone	email	address
1	Arun Kumar S	9876543210	arun@gmail.com	Gudalur
2	Pritya Devi	9876543211	pritya@gmail.com	Chennai
3	Rahul Raj	9876543212	rahul@gmail.com	Coimbatore
4	Karthik S	9876543213	karthik@gmail.com	Salem
5	Divya M	9876543214	divya@gmail.com	Erode
6	Suresh Kumar	9876543215	suresh@gmail.com	Tiruppur
7	Meena R	9876543216	meena@gmail.com	Ooty
8	Vignesh K	9876543217	vignesh@gmail.com	Madurai

The Output pane shows the following actions:

#	Time	Action	Message	Duration / Fetch
1	13:45:10	USE banking_db	0 row(s) affected	0.015 sec
2	13:45:10	SHOW TABLES	4 row(s) returned	0.047 sec / 0.000 sec
3	13:47:34	SELECT * FROM Customer LIMIT 0, 1000	8 row(s) returned	0.046 sec / 0.000 sec

Screenshot 3 – AccountType Table



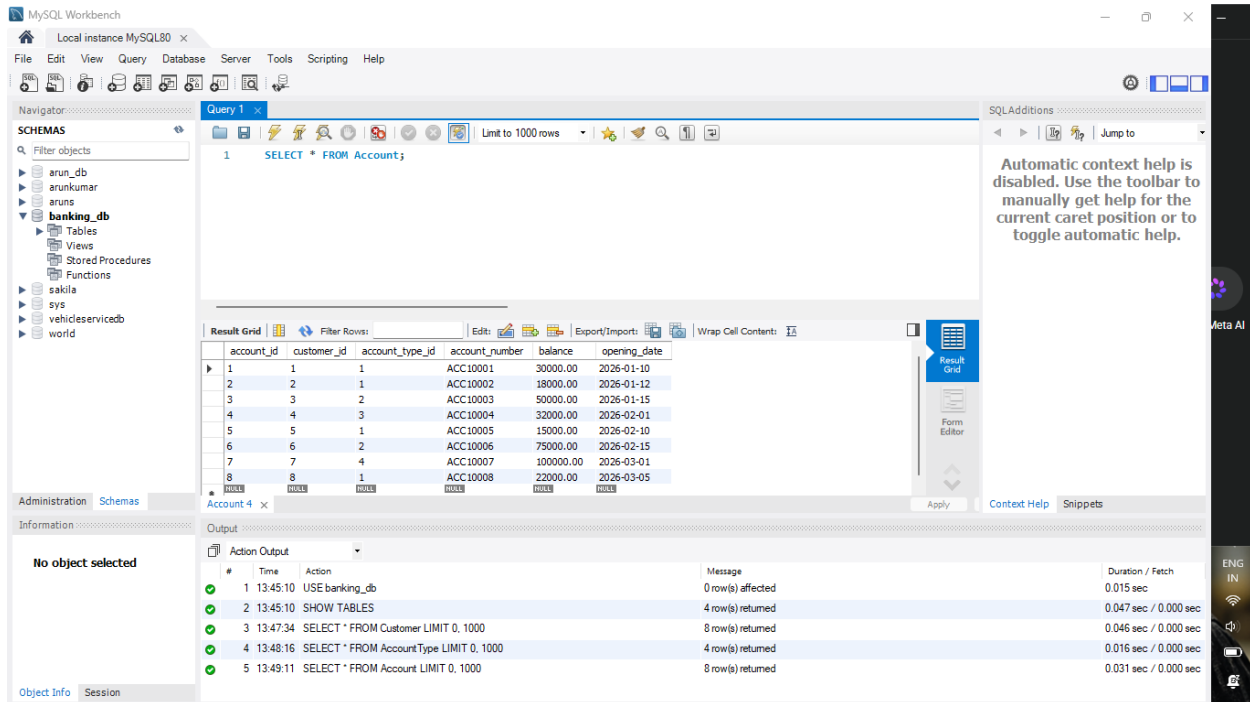
MySQL Workbench interface showing the AccountType table data. The query executed is `SELECT * FROM AccountType;`. The result grid displays 4 rows of account type information.

account_type_id	account_type_name	minimum_balance
1	Savings	1000.00
2	Current	5000.00
3	Salary	0.00
4	Fixed Deposit	10000.00

The Output pane shows the following actions:

#	Time	Action	Message	Duration / Fetch
1	13:45:10	USE banking_db	0 row(s) affected	0.015 sec
2	13:45:10	SHOW TABLES	4 row(s) returned	0.047 sec / 0.000 sec
3	13:47:34	SELECT * FROM Customer LIMIT 0, 1000	8 row(s) returned	0.046 sec / 0.000 sec
4	13:48:16	SELECT * FROM AccountType LIMIT 0, 1000	4 row(s) returned	0.016 sec / 0.000 sec

Screenshot 4 – Account Table



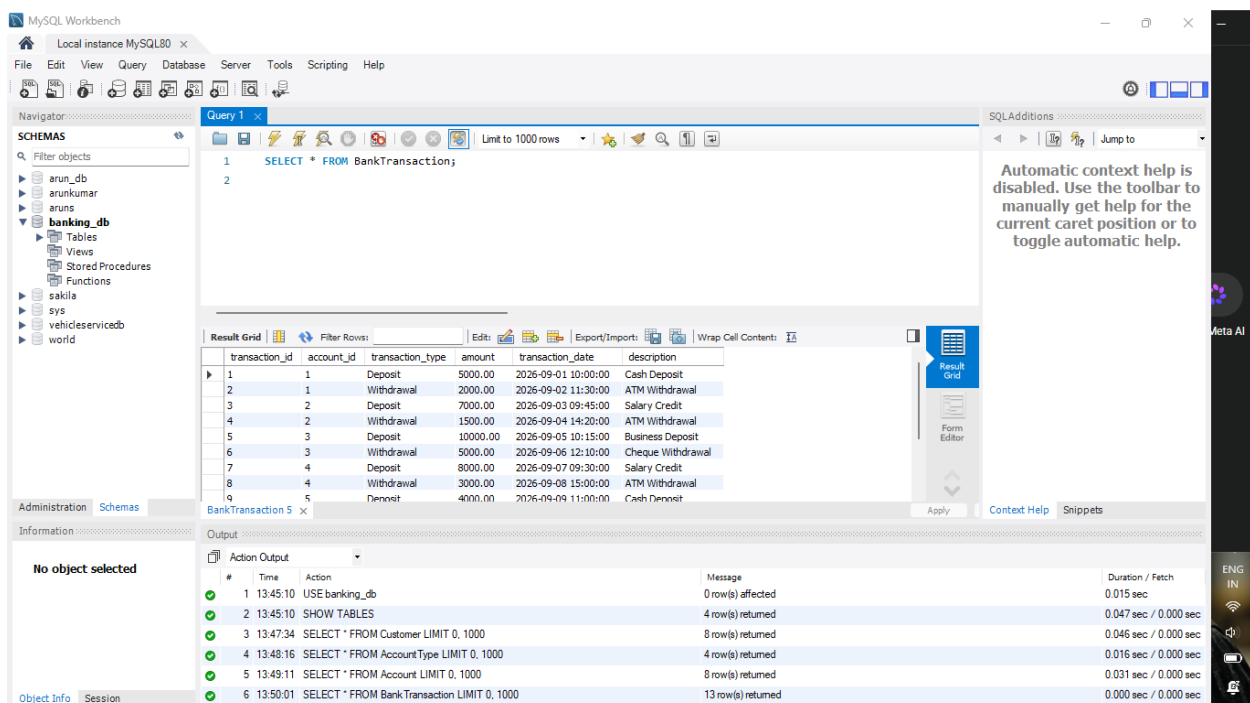
MySQL Workbench interface showing the 'Account' table in the 'banking_db' database. The table structure is as follows:

account_id	customer_id	account_type_id	account_number	balance	opening_date
1	1	1	ACC10001	30000.00	2026-01-10
2	2	1	ACC10002	18000.00	2026-01-12
3	3	2	ACC10003	50000.00	2026-01-15
4	4	3	ACC10004	32000.00	2026-02-01
5	5	1	ACC10005	15000.00	2026-02-10
6	6	2	ACC10006	75000.00	2026-02-15
7	7	4	ACC10007	100000.00	2026-03-01
8	8	1	ACC10008	22000.00	2026-03-05

The 'Output' pane shows the following actions:

#	Time	Action	Message	Duration / Fetch
1	13:45:10	USE banking_db	0 row(s) affected	0.015 sec
2	13:45:10	SHOW TABLES	4 row(s) returned	0.047 sec / 0.000 sec
3	13:47:34	SELECT * FROM Customer LIMIT 0, 1000	8 row(s) returned	0.046 sec / 0.000 sec
4	13:48:16	SELECT * FROM AccountType LIMIT 0, 1000	4 row(s) returned	0.016 sec / 0.000 sec
5	13:49:11	SELECT * FROM Account LIMIT 0, 1000	8 row(s) returned	0.031 sec / 0.000 sec

Screenshot 5 – Transaction Table



MySQL Workbench interface showing the 'BankTransaction' table in the 'banking_db' database. The table structure is as follows:

transaction_id	account_id	transaction_type	amount	transaction_date	description
1	1	Deposit	5000.00	2026-09-01 10:00:00	Cash Deposit
2	1	Withdrawal	2000.00	2026-09-02 11:30:00	ATM Withdrawal
3	2	Deposit	7000.00	2026-09-03 09:45:00	Salary Credit
4	2	Withdrawal	1500.00	2026-09-04 14:20:00	ATM Withdrawal
5	3	Deposit	10000.00	2026-09-05 10:15:00	Business Deposit
6	3	Withdrawal	5000.00	2026-09-06 12:10:00	Cheque Withdrawal
7	4	Deposit	8000.00	2026-09-07 09:30:00	Salary Credit
8	4	Withdrawal	3000.00	2026-09-08 15:00:00	ATM Withdrawal
9	5	Deposit	4000.00	2026-09-09 11:00:00	Cash Deposit

The 'Output' pane shows the following actions:

#	Time	Action	Message	Duration / Fetch
1	13:45:10	USE banking_db	0 row(s) affected	0.015 sec
2	13:45:10	SHOW TABLES	4 row(s) returned	0.047 sec / 0.000 sec
3	13:47:34	SELECT * FROM Customer LIMIT 0, 1000	8 row(s) returned	0.046 sec / 0.000 sec
4	13:48:16	SELECT * FROM AccountType LIMIT 0, 1000	4 row(s) returned	0.016 sec / 0.000 sec
5	13:49:11	SELECT * FROM Account LIMIT 0, 1000	8 row(s) returned	0.031 sec / 0.000 sec
6	13:50:01	SELECT * FROM BankTransaction LIMIT 0, 1000	13 row(s) returned	0.000 sec / 0.000 sec

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL query:

```
1 INSERT INTO BankTransaction
2 (account_id, transaction_type, amount, description)
3 VALUES
4 (1, 'Deposit', 5000.00, 'Cash Deposit');
```

The 'Output' tab shows the execution results:

#	Time	Action	Message	Duration / Fetch
2	13:45:10	SHOW TABLES	4 row(s) returned	0.047 sec / 0.000 sec
3	13:47:34	SELECT * FROM Customer LIMIT 0, 1000	8 row(s) returned	0.045 sec / 0.000 sec
4	13:48:16	SELECT * FROM AccountType LIMIT 0, 1000	4 row(s) returned	0.016 sec / 0.000 sec
5	13:49:11	SELECT * FROM Account LIMIT 0, 1000	8 row(s) returned	0.031 sec / 0.000 sec
6	13:50:01	SELECT * FROM BankTransaction LIMIT 0, 1000	13 row(s) returned	0.000 sec / 0.000 sec
7	13:50:55	INSERT INTO BankTransaction (account_id, transaction_type, amount, description) V...	1 row(s) affected	0.062 sec

Screenshot 6 – SELECT and Filtering

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL queries:

```
4 SELECT * FROM Account
5 ORDER BY balance DESC;
7 SELECT * FROM Customer
8 WHERE customer_name LIKE 'A%';
10 SELECT * FROM BankTransaction
11 WHERE amount BETWEEN 2000 AND 10000;
```

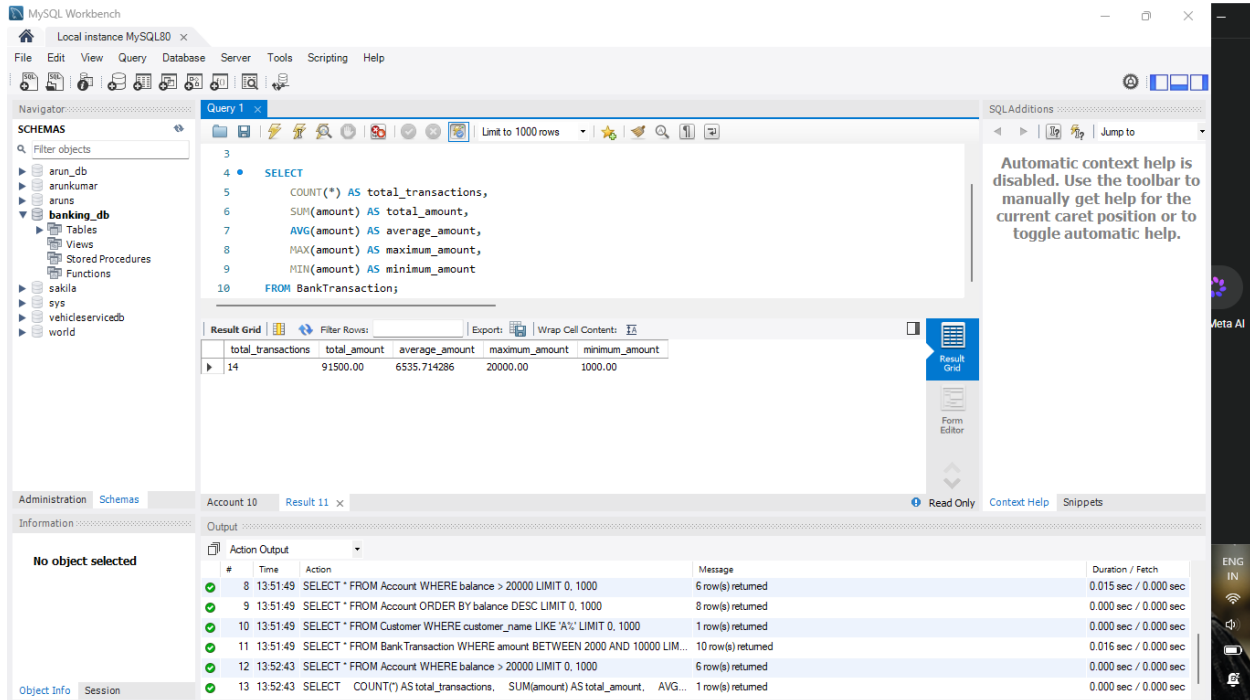
The 'Result Grid' shows the results of the first query:

transaction_id	account_id	transaction_type	amount	transaction_date	description
1	1	Deposit	5000.00	2026-09-01 10:00:00	Cash Deposit
2	1	Withdrawal	2000.00	2026-09-02 11:30:00	ATM Withdrawal
3	2	Deposit	7000.00	2026-09-03 09:45:00	Salary Credit
5	3	Deposit	10000.00	2026-09-05 10:15:00	Business Deposit
6	3	Withdrawal	5000.00	2026-09-06 12:10:00	Cheque Withdrawal
7	4	Deposit	8000.00	2026-09-07 09:30:00	Salary Credit
8	4	Withdrawal	3000.00	2026-09-08 15:00:00	ATM Withdrawal
9	5	Deposit	4000.00	2026-09-09 11:00:00	Cash Deposit
12	6	Withdrawal	5000.00	2026-09-12 13:15:00	Bank Withdrawal

The 'Output' tab shows the execution results:

#	Time	Action	Message	Duration / Fetch
6	13:50:01	SELECT * FROM BankTransaction LIMIT 0, 1000	13 row(s) returned	0.000 sec / 0.000 sec
7	13:50:55	INSERT INTO BankTransaction (account_id, transaction_type, amount, description) V...	1 row(s) affected	0.062 sec
8	13:51:49	SELECT * FROM Account WHERE balance > 20000 LIMIT 0, 1000	6 row(s) returned	0.015 sec / 0.000 sec
9	13:51:49	SELECT * FROM Account ORDER BY balance DESC LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
10	13:51:49	SELECT * FROM Customer WHERE customer_name LIKE 'A%' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
11	13:51:49	SELECT * FROM BankTransaction WHERE amount BETWEEN 2000 AND 10000 LIM...	10 row(s) returned	0.016 sec / 0.000 sec

Screenshot 7 – Aggregate Functions



MySQL Workbench interface showing a query using aggregate functions. The query is as follows:

```

3
4 SELECT
5     COUNT(*) AS total_transactions,
6     SUM(amount) AS total_amount,
7     AVG(amount) AS average_amount,
8     MAX(amount) AS maximum_amount,
9     MIN(amount) AS minimum_amount
10    FROM BankTransaction;

```

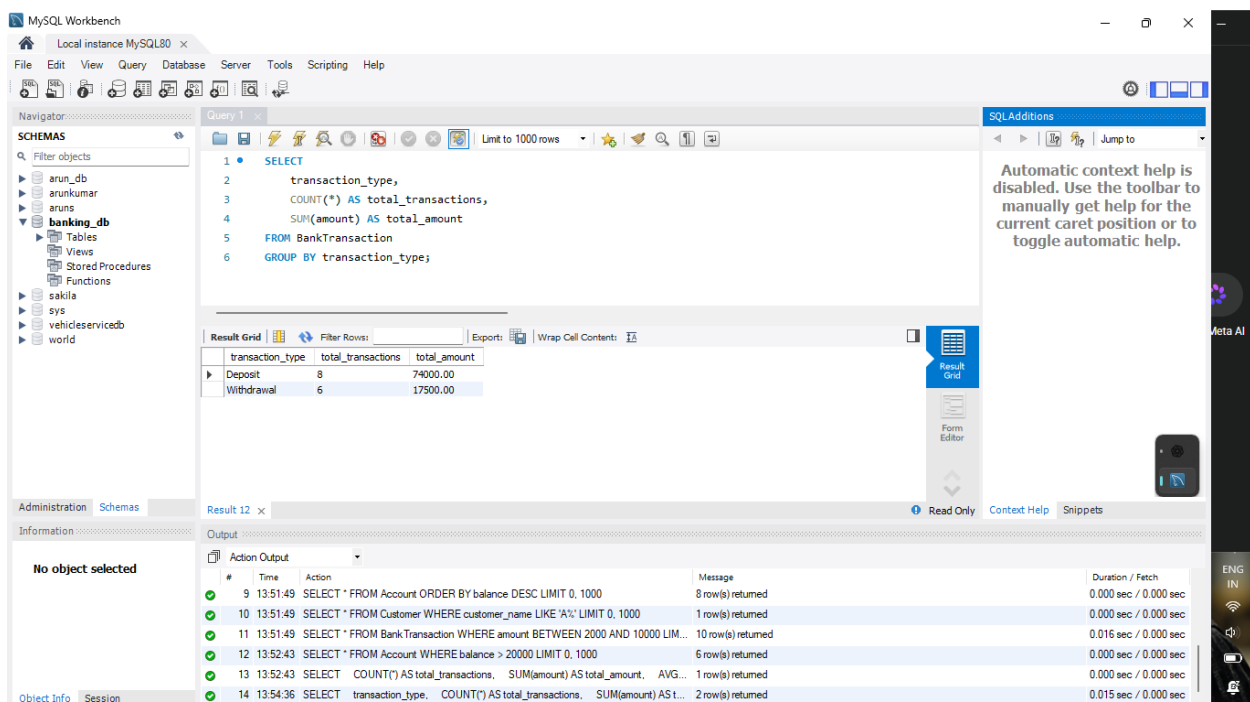
The result grid shows the following data:

	total_transactions	total_amount	average_amount	maximum_amount	minimum_amount
14	14	91500.00	6535.714286	20000.00	1000.00

The bottom panel shows the Action Output with the following details:

#	Time	Action	Message	Duration / Fetch
8	13:51:49	SELECT * FROM Account WHERE balance > 20000 LIMIT 0, 1000	6 row(s) returned	0.015 sec / 0.000 sec
9	13:51:49	SELECT * FROM Account ORDER BY balance DESC LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
10	13:51:49	SELECT * FROM Customer WHERE customer_name LIKE 'A%' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
11	13:51:49	SELECT * FROM BankTransaction WHERE amount BETWEEN 2000 AND 10000 LIM...	10 row(s) returned	0.016 sec / 0.000 sec
12	13:52:43	SELECT * FROM Account WHERE balance > 20000 LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
13	13:52:43	SELECT COUNT(*) AS total_transactions, SUM(amount) AS total_amount, AVG...	1 row(s) returned	0.000 sec / 0.000 sec

Screenshot 8 – GROUP BY



MySQL Workbench interface showing a query using GROUP BY. The query is as follows:

```

1 SELECT
2     transaction_type,
3     COUNT(*) AS total_transactions,
4     SUM(amount) AS total_amount
5    FROM BankTransaction
6   GROUP BY transaction_type;

```

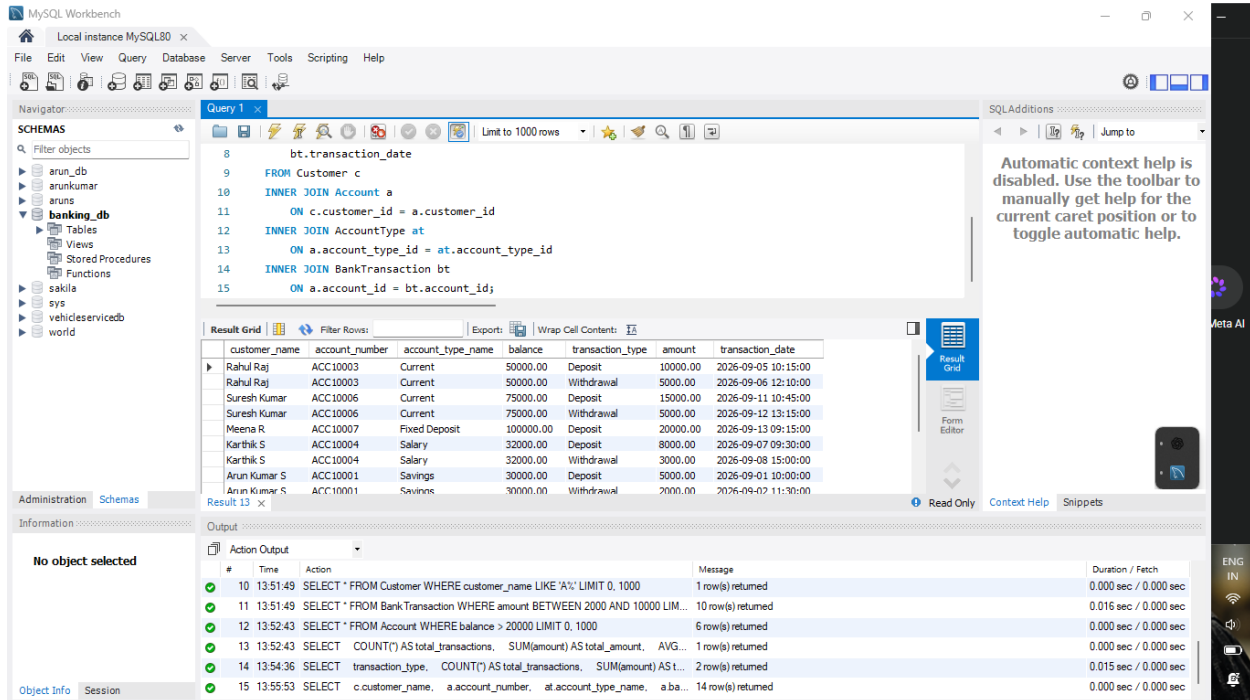
The result grid shows the following data:

	transaction_type	total_transactions	total_amount
9	Deposit	8	74000.00
10	Withdrawal	6	17500.00

The bottom panel shows the Action Output with the following details:

#	Time	Action	Message	Duration / Fetch
9	13:51:49	SELECT * FROM Account ORDER BY balance DESC LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
10	13:51:49	SELECT * FROM Customer WHERE customer_name LIKE 'A%' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
11	13:51:49	SELECT * FROM BankTransaction WHERE amount BETWEEN 2000 AND 10000 LIM...	10 row(s) returned	0.016 sec / 0.000 sec
12	13:52:43	SELECT * FROM Account WHERE balance > 20000 LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
13	13:52:43	SELECT COUNT(*) AS total_transactions, SUM(amount) AS total_amount, AVG...	1 row(s) returned	0.000 sec / 0.000 sec
14	13:54:36	SELECT transaction_type, COUNT(*) AS total_transactions, SUM(amount) AS t...	2 row(s) returned	0.015 sec / 0.000 sec

Screenshot 9 – INNER JOIN



MySQL Workbench interface showing a query with INNER JOINs and its results.

Query 1:

```

8      bt.transaction_date
9      FROM Customer c
10     INNER JOIN Account a
11     ON c.customer_id = a.customer_id
12     INNER JOIN AccountType at
13     ON a.account_type_id = at.account_type_id
14     INNER JOIN BankTransaction bt
15     ON a.account_id = bt.account_id;

```

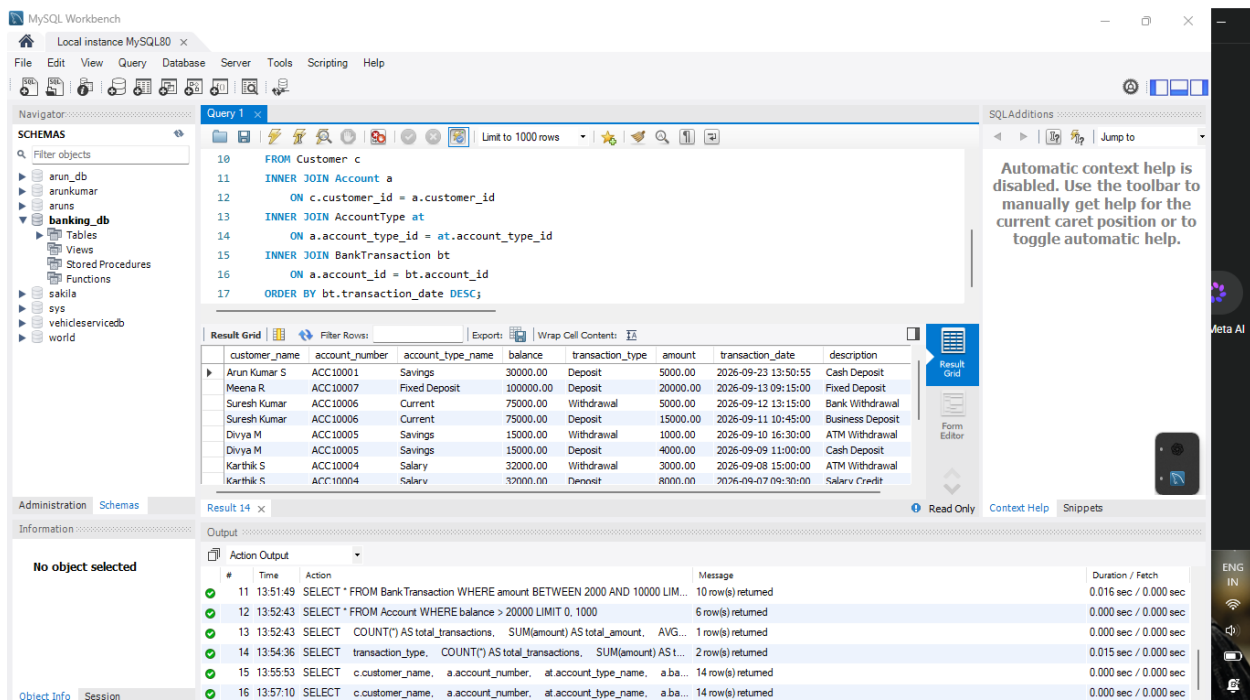
Result Grid:

customer_name	account_number	account_type_name	balance	transaction_type	amount	transaction_date
Rahul Raj	ACC10003	Current	50000.00	Deposit	10000.00	2026-09-05 10:15:00
Rahul Raj	ACC10003	Current	50000.00	Withdrawal	5000.00	2026-09-06 12:10:00
Suresh Kumar	ACC10006	Current	75000.00	Deposit	15000.00	2026-09-11 10:45:00
Suresh Kumar	ACC10006	Current	75000.00	Withdrawal	5000.00	2026-09-12 13:15:00
Meena R	ACC10007	Fixed Deposit	100000.00	Deposit	20000.00	2026-09-13 09:15:00
Karthik S	ACC10004	Salary	32000.00	Deposit	8000.00	2026-09-07 09:30:00
Karthik S	ACC10004	Salary	32000.00	Withdrawal	3000.00	2026-09-08 15:00:00
Arun Kumar S	ACC10001	Savings	30000.00	Deposit	5000.00	2026-09-01 10:00:00
Arun Kumar S	ACC10001	Savings	30000.00	Withdrawal	2000.00	2026-09-02 11:30:00

Output:

#	Time	Action	Message	Duration / Fetch
10	13:51:49	SELECT * FROM Customer WHERE customer_name LIKE 'A%' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
11	13:51:49	SELECT * FROM BankTransaction WHERE amount BETWEEN 2000 AND 10000 LIMIT 0, 1000	10 row(s) returned	0.016 sec / 0.000 sec
12	13:52:43	SELECT * FROM Account WHERE balance > 20000 LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
13	13:52:43	SELECT COUNT(*) AS total_transactions, SUM(amount) AS total_amount, AVG...	1 row(s) returned	0.000 sec / 0.000 sec
14	13:54:36	SELECT transaction_type, COUNT(*) AS total_transactions, SUM(amount) AS t...	2 row(s) returned	0.015 sec / 0.000 sec
15	13:55:53	SELECT c.customer_name, a.account_number, at.account_type_name, a.ba...	14 row(s) returned	0.000 sec / 0.000 sec

Screenshot 10 – Final Database / Query Results



MySQL Workbench interface showing a query with INNER JOINs and its results.

Query 14:

```

10     FROM Customer c
11     INNER JOIN Account a
12     ON c.customer_id = a.customer_id
13     INNER JOIN AccountType at
14     ON a.account_type_id = at.account_type_id
15     INNER JOIN BankTransaction bt
16     ON a.account_id = bt.account_id
17     ORDER BY bt.transaction_date DESC;

```

Result Grid:

customer_name	account_number	account_type_name	balance	transaction_type	amount	transaction_date	description
Arun Kumar S	ACC10001	Savings	30000.00	Deposit	5000.00	2026-09-23 13:50:55	Cash Deposit
Meena R	ACC10007	Fixed Deposit	100000.00	Deposit	20000.00	2026-09-13 09:15:00	Fixed Deposit
Suresh Kumar	ACC10006	Current	75000.00	Withdrawal	5000.00	2026-09-12 13:15:00	Bank Withdrawal
Suresh Kumar	ACC10006	Current	75000.00	Deposit	15000.00	2026-09-11 10:45:00	Business Deposit
Divya M	ACC10005	Savings	15000.00	Withdrawal	1000.00	2026-09-10 16:30:00	ATM Withdrawal
Divya M	ACC10005	Savings	15000.00	Deposit	4000.00	2026-09-09 11:00:00	Cash Deposit
Karthik S	ACC10004	Salary	32000.00	Withdrawal	3000.00	2026-09-08 15:00:00	ATM Withdrawal
Karthik S	ACC10004	Salary	32000.00	Deposit	8000.00	2026-09-07 09:30:00	Salary Credit

Output:

#	Time	Action	Message	Duration / Fetch
11	13:51:49	SELECT * FROM BankTransaction WHERE amount BETWEEN 2000 AND 10000 LIMIT 0, 1000	10 row(s) returned	0.016 sec / 0.000 sec
12	13:52:43	SELECT * FROM Account WHERE balance > 20000 LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
13	13:52:43	SELECT COUNT(*) AS total_transactions, SUM(amount) AS total_amount, AVG...	1 row(s) returned	0.000 sec / 0.000 sec
14	13:54:36	SELECT transaction_type, COUNT(*) AS total_transactions, SUM(amount) AS t...	2 row(s) returned	0.015 sec / 0.000 sec
15	13:55:53	SELECT c.customer_name, a.account_number, at.account_type_name, a.ba...	14 row(s) returned	0.000 sec / 0.000 sec
16	13:57:10	SELECT c.customer_name, a.account_number, at.account_type_name, a.ba...	14 row(s) returned	0.000 sec / 0.000 sec

5. Technical Explanation – 5 Marks

Explain the following:

Database Design

Customer stores personal information. AccountType stores account categories such as Savings and Current. Account stores account number, customer, account type, opening date, and balance. Transaction stores deposits and withdrawals with date, amount, and description. Separating the tables reduces duplicate data and improves consistency and maintainability.

Keys and Constraints

Primary keys identify each record uniquely. Foreign keys connect related tables. NOT NULL ensures required data is present, UNIQUE prevents duplicate values, AUTO_INCREMENT creates IDs automatically, and DEFAULT sets an initial balance or standard value.

SQL Operations

INSERT creates records, SELECT retrieves records, UPDATE modifies records, and DELETE removes records. These form the basic CRUD operations.

Data Retrieval

WHERE filters records, ORDER BY sorts results, LIKE searches matching text patterns, and BETWEEN retrieves values within a specified range.

Aggregate Functions and Joins

COUNT counts records, SUM calculates totals, AVG calculates averages, MAX finds the highest value, and MIN finds the lowest value. INNER JOIN combines matching records from related tables using primary and foreign keys.

6. Conclusion – 5 Marks

Conclusion

Summarize the database developed, the major features implemented, SQL operations performed, and the concepts learned through the project.

Future Enhancements

Mention possible future improvements, such as:

- • Online fund transfer and transaction facilities.
- • Automated transaction and balance notifications.
- • Detailed transaction history and account statements.
- • Automatic bank statement generation.
- • Loan and account management.
- • Integration with web and mobile banking applications.
- • Automated database backup and recovery.